

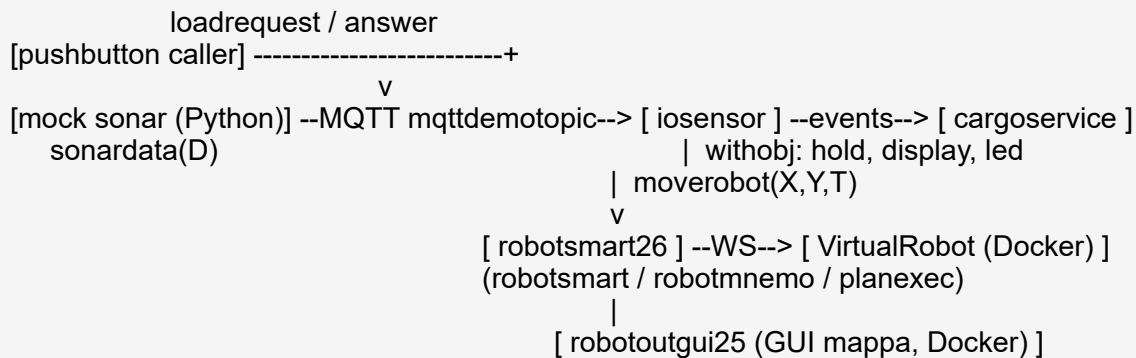
TemaFinale26 – Project (Progetto)

Scopo della fase

Questa fase traduce il modello logico ([Sprint 1 – Analisi del Problema](#)) in un **progetto concreto**: architettura di deployment, struttura del progetto, realizzazione dei componenti e tecnologie. Le decisioni qui prese guidano la fase di **Development**.

Architettura del sistema

Il sistema e' un insieme di (micro)servizi a scambio di messaggi. Il **cargoservice** e' un client del servizio **robotsmart26** (gia' fornito dal docente) e riceve i dati del sonar dal PicoW, qui **sostituito dal mock Python**.



Broker MQTT: mosquitto (Docker). cargoservice e robotsmart26 girano come processi sul PC host.

Componente	Realizzazione	Trasporto
cargoservice	QActor (da costruire)	TCP/CoAP + MQTT
iosensor	QActor (da costruire)	MQTT (subscribe sonar)
robotsmart26	servizio del docente (gia' pronto)	TCP porta 8020
VirtualRobot + GUI	immagini Docker del docente	WS 8090/8091/8085
sonar (PicoW)	mockPicowRadar.py	MQTT 1883
pushbutton	caller (client qak/Java)	TCP/CoAP
display / LED / marker	oggetti simulati (withobj / attore)	in-process

Struttura del progetto (prevista)

cargoservice26/	
src/cargoservice26.qak	modello qak (da generare con il plugin QAK)
src/it/unibo/cargoservice/Cargoservice.kt	codice generato
src/it/unibo/iosensor/Iosensor.kt	codice generato
src/it/unibo/ctxcargo/MainCtxcargo.kt	entry point generato
cargoservice26.pl sysRules.pl	teorie Prolog generate
utils/domain/Hold.java	stato della stiva (slot1..5)

utils/devices/DisplaySim.java	display simulato (output a video)
utils/devices/LedSim.java	LED simulato (lampeggio a video)
build.gradle settings.gradle	build (come robotsmart26usage)

Il progetto e' modellato su `robotsmart26usage` (stesso pattern: client che dichiara `ExternalQActor robotsmart context ctxrobotsmart` e lo comanda con `moverobot`).

Realizzazione dei componenti

cargoservice (QActor)

Implementa la FSM dell'analisi del problema. Incapsula tre oggetti con `withobj`: `hold` (stato stiva), `display` e `led`. Comanda il robot con `request robotsmart -m moverobot` e reagisce a `containerAtIOPort`, `markingDone`, `sonarFail/sonarOk`, e al timeout `whenTime 30000`.

iosensor (QActor)

Si iscrive al sonar (mock via MQTT, topic `mqttdemotopic`, messaggio `sonardata(D)`) e applica i filtri temporali del committente, emettendo:

```
D < DFREE/2 per ~3 s    -> emit containerAtIOPort
D > DFREE   per ≥ 3 s   -> emit sonarFail
D ≤ DFREE                -> emit sonarOk
```

Così il `cargoservice` resta indipendente dalla sorgente: sostituendo il mock col `PicoW` reale (stesso messaggio `sonardata(D)`) nulla cambia.

Hold (dominio)

```
public class Hold {
    boolean ioportOccupied();
    boolean isFull();           // slot1..4 occupati
    String reserveFreeSlot();    // nome slot libero, "" se pieno
    void confirmStored(String s);
    void freeSlot(String s);
    int slotX(String s); int slotY(String s); // posizioni dalla mappa
}
```

DisplaySim e LedSim (dispositivi simulati)

`DisplaySim.show(msg)` stampa a video lo stato/risposta; `LedSim.blink(on)` avvia/ferma un lampeggio simulato (timer a video). Sostituibili con dispositivi reali senza modificare il `cargoservice`.

Posizioni e mappa

Le posizioni-obiettivo per moverobot(X,Y) provengono dal materiale del docente (robotsmart26/utils/callers/Robotsmart26Cmds.java):

```
IOPort : moverobot(4,0)    //"port position" (commento del docente)
slot   : moverobot(1,1) | moverobot(1,4) | moverobot(3,2) | moverobot(5,3)
```

Da confermare (D1). La figura del tema mostra il layout (HOME, SONAR, SLOT1..5, IOPORT) ma non le coordinate esatte; il mapping preciso slot1..5 ↔ posizioni e la posizione di **slot5** vanno confermati sulla mappa del docente (cfr. tf25map). Fino ad allora le posizioni sopra sono usate come parametri configurabili, non come valori fissi nel codice.

Deployment

- 1) Docker: VirtualRobot + GUI mappa + mosquitto
docker compose -f robotsmart26/yamls/unibobasic26.yaml up
- 2) robotsmart26 (servizio del docente) gradlew run (porta 8020)
- 3) cargoservice26 (questo progetto) gradlew run (porta ctxcargo)
- 4) mock sonar python mockPicowRadar.py --broker <IP>
- 5) pushbutton caller invia loadrequest al cargoservice

Nota sull'indirizzo MQTT (cfr. robotsmart26tf26.qak del docente): con mosquitto in Docker, i client che si iscrivono via localhost possono non ricevere i messaggi; si usa quindi l'**IP reale** del PC (es. quello del Wi-Fi) sia nel modello qak sia nel mock.

Verso Development e Prototypes

La fase di **Development** produrrà cargoservice26.qak e gli oggetti Java (Hold, DisplaySim, LedSim, iosensor), genererà il codice con il plugin QAK e lo eseguirà nello scenario sopra. La fase **Prototypes** eseguirà il **Test Plan** dell'analisi del problema (caso nominale come demo significativa, più i casi di rifiuto, timeout e out-of-service).



By Alexander Kreuzer

Email: Alexander.kreuzer@studio.unibo.it

GIT repo: https://github.com/Alexing-uni/ISS26_Alexander_Kreuzer